© The Author(s), under exclusive license to APress Media, LLC, part of Springer Nature 2025
S. K. Sahu, *Generative AI-Driven Application Development with Java*
https://doi.org/10.1007/979-8-8688-1609-3_7

7. From Chatty to Clever: Retrieval-Augmented Generation

Satej Kumar Sahu[1]
(1) Berhampur, Odisha, India

The goal of this chapter is to use contextual information that is more specific to organizations in order to provide context-specific answers to customer questions. This will be introduced by using LangChain4J with retrieval-augmented generation (RAG).

Introduction

In the previous chapter, you learned about using LangChain4j to integrate your Java application with OpenAI to get information related to different aircrafts. This scenario is useful in many cases when you want generalized information. However, customer problems and queries are more often contextual than not to the organization they are interacting with. For example, our imaginary travel company TravelX uses a specific set of aircrafts for travel. In the subsequent sections, you will learn how to leverage contextual information that might be present in PDFs, files, and other systems internal to the organization to give contextual answers to customer queries. Along the way, you will learn about the RAG pattern which helps to find contextual information. There are many components of the RAG paradigm like embeddings and vector stores that we will touch upon while integrating them into your example TravelX application.

Contextuality in the LLM Ecosystem

If we refer to Figure 7-1, the customer query reaches our Spring Boot application and then is redirected to OpenAI. The response from the LLM is shared with our customers. This simple pattern is helpful in many cases but may not always be up-to-date with the latest information given the model might have been trained only up to a certain point in the past. In the information age of today, current and factual information always helps to serve our customers better. In addition, a company's edge over its competitors is its contextual in-house proprietary data along with any customer-specific data that it has. So, if we ask the application anything particular to our application, without any prompt hinting, it may try to hallucinate. One way to reduce hallucination is to use prompt engineering. You will start with this and incrementally learn and see the usage of the RAG paradigm.

To set the context for your use case, you now want to provide information to your customers about the flights that are operational through our airline TravelX. As per the previous chapter, you were able to get some information related to Aircraft and share the JSON of the information as a response. Since this is a new requirement for your team and you need to share the flight information, you will start with creating foundational entities for the flight details.

Similar to the Aircraft class, you will create a POJO for the Flight entity. The flight details that you will be sharing with our customers will have a few details like aircraft model, date of departure, from city, to city, departure airport, arrival airport, flight number, departure time, arrival time, and status of the flight. Figure 7-1 depicts the UML diagram for all the classes and relationships you will using throughout this chapter.

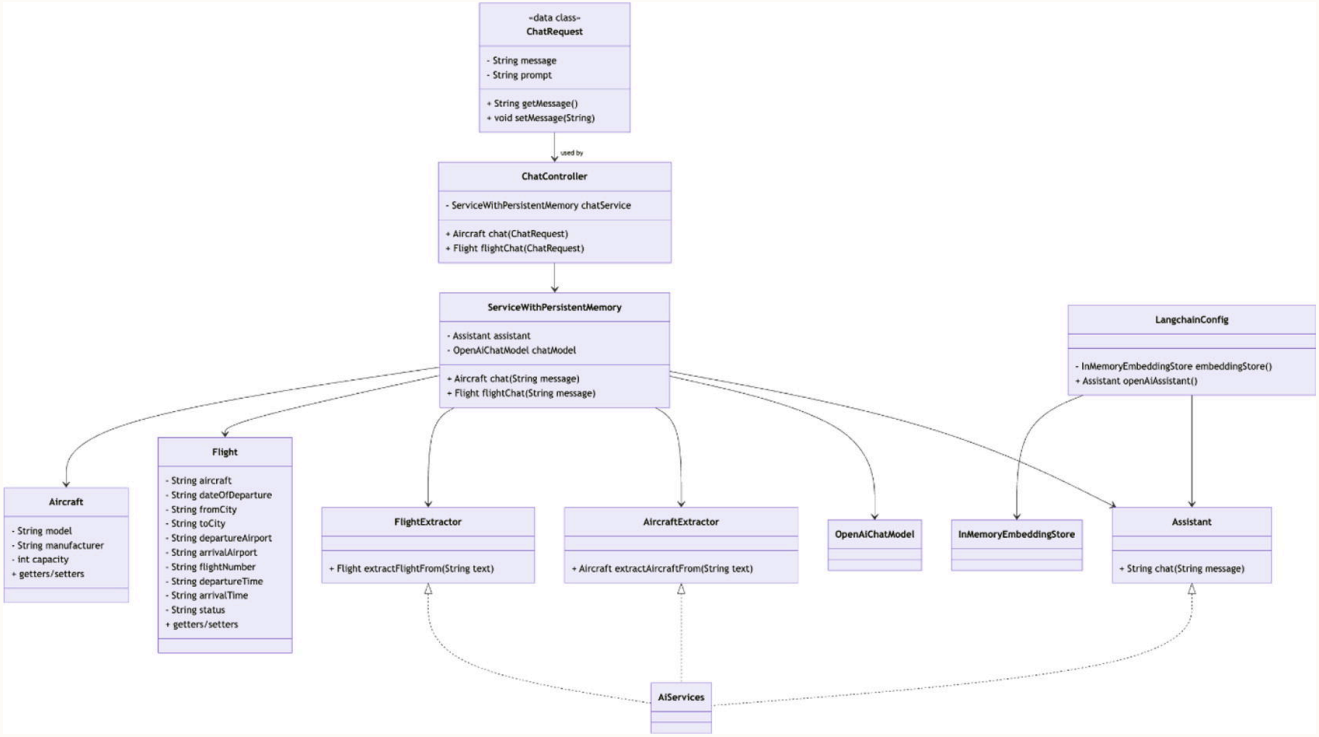


Figure 7-1 UML diagram of the classes and relationships of project

File: ./src/main/java/com/example/travelx/Flight.java

```
package com.example.travelx;
public class Flight {
   private String aircraft;
   private String dateOfDeparture;
   private String fromCity;
   private String toCity;
   private String departureAirport;
   private String arrivalAirport;
   private String flightNumber;
   private String departureTime;
   private String arrivalTime;
   private String status;
   // Getters and Setters
  public String getAircraft() {
      return aircraft;
   }
   public void setAircraft(String aircraft) {
      this.aircraft = aircraft;
   }
   public String getDateOfDeparture() {
      return dateOfDeparture;
   }
   public void setDateOfDeparture(String dateOfDeparture) {
      this.dateOfDeparture = dateOfDeparture;
   }
   public String getFromCity() {
      return fromCity;
   }
   public void setFromCity(String fromCity) {
      this.fromCity = fromCity;
   }
   public String getToCity() {
      return toCity;
   }
   public void setToCity(String toCity) {
      this.toCity = toCity;
   }
   public String getDepartureAirport() {
      return departureAirport;
   }
   public void setDepartureAirport(String departureAirport) {
```

```
        this.departureAirport = departureAirport;
    }
    public String getArrivalAirport() {
        return arrivalAirport;
    }
    public void setArrivalAirport(String arrivalAirport) {
        this.arrivalAirport = arrivalAirport;
    }
    public String getFlightNumber() {
        return flightNumber;
    }
    public void setFlightNumber(String flightNumber) {
        this.flightNumber = flightNumber;
    }
    public String getDepartureTime() {
        return departureTime;
    }
    public void setDepartureTime(String departureTime) {
        this.departureTime = departureTime;
    }
    public String getArrivalTime() {
        return arrivalTime;
    }
    public void setArrivalTime(String arrivalTime) {
        this.arrivalTime = arrivalTime;
    }
    public String getStatus() {
        return status;
    }
    public void setStatus(String status) {
        this.status = status;
    }
    @Override
    public String toString() {
        return "Flight {" +
                " aircraft = \"" + aircraft + "\"" +
                ", dateOfDeparture = \"" + dateOfDeparture + "\"" +
                ", fromCity = \"" + fromCity + "\"" +
                ", toCity = \"" + toCity + "\"" +
                ", departureAirport = \"" + departureAirport + "\"" +
                ", arrivalAirport = \"" + arrivalAirport + "\"" +
                ", flightNumber = \"" + flightNumber + "\"" +
                ", departureTime = \"" + departureTime + "\"" +
                ", arrivalTime = \"" + arrivalTime + "\"" +
                ", status = \"" + status + "\"" +
                " }";
    }
}
```

The goal of setting up the previous entity class is to set the structure to extract the LLM response into a JSON format that you can share as response with the customer. The next step is to create the interface similar to AircraftExtractor that you created that LangChain4j uses to extract the structure from the LLM response.

File: ./src/main/java/com/example/travelx/FlightExtractor.java

```
package com.example.travelx;
public interface FlightExtractor {
    Flight extractFlightFrom(String text);
}
```

Now, you will create a new method called flightChat in the service file to get the flight details.

File: ./src/main/java/com/example/travelx/ServiceWithPersistentMemory.java

```
package com.example.travelx;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
```

```
import dev.langchain4j.model.openai.OpenAiChatModel;
import dev.langchain4j.service.AiServices;
@Configuration
public class ServiceWithPersistentMemory {
    private Assistant assistant;
    private OpenAiChatModel chatModel;
    @Bean
    public Assistant chatAssistant(Assistant assistant, OpenAiChatModel chatModel) {
        this.assistant = assistant;
        this.chatModel = chatModel;
        return this.assistant;
    }
    public Aircraft chat(String message) {
        String response = this.assistant.chat(message);
        AircraftExtractor aircraftExtractor = AiServices.create(AircraftExtractor.class, chatModel);
        Aircraft aircraft = aircraftExtractor.extractAircraftFrom(response);
        return aircraft;
    }
    public Flight flightChat(String message) {
        String response = this.assistant.chat(message);
        FlightExtractor flightExtractor = AiServices.create(FlightExtractor.class, chatModel);
        Flight flight = flightExtractor.extractFlightFrom(response);
        return flight;
    }
}
```

The final step to invoke this service method is through the controller where you will create a new route for the path /api/flight-chat.

File: ./src/main/java/com/example/travelx/ChatController.java

```
package com.example.travelx;
import org.springframework.web.bind.annotation.*;
@RestController
public class ChatController {
    private ServiceWithPersistentMemory chatService;
    public ChatController(ServiceWithPersistentMemory chatService) {
        this.chatService = chatService;
    }
    @PostMapping("/api/chat")
    public Aircraft chat(@RequestBody ChatRequest chatRequest) {
        return this.chatService.chat(chatRequest.getMessage());
    }
    @PostMapping("/api/flight-chat")
    public Flight flightChat(@RequestBody ChatRequest chatRequest) {
        return this.chatService.flightChat(chatRequest.getMessage());
    }
}
```

Let's power up the application and see the response from this API for a flight that is not available from any airlines in the world at this moment apart from your airline.

```
curl -X POST http://localhost:8080/api/flight-chat -H "Content-Type: application/json" -d '{
 "message": "Tell me about flight details for AABB101."
}'
```

Output:

```
{"aircraft":"Boeing 737","dateOfDeparture":"2023-10-15","fromCity":"New York","toCity":"Los Angeles","departureAirport":"JFK","arrivalAirport":"LAX","flightN
```

Note Please make sure to delete the chat-memory.db file to remove any historical chat from previous chapters.

The previous is a clear case of hallucination since this flight does not exist in the knowledge base of the trained LLM nor in any real-world scenario. Also, if you observe the output response, the flightNumber is different than requested. Essentially the LLM was not able to find the requested flight and instead shared info for some other flight data.

Previously, you had a component to add some prompt engineering in the file ChatRequest.java to check for factual responses. Considering its presence, you should not be receiving this response. Let's modify the code slightly to take this into consideration.

File: ./src/main/java/com/example/travelx/ChatRequest.java

```
package com.example.travelx;
class ChatRequest {
   private String message;
   public static String defaultMessage = "Sorry, I am not aware of this context. Please let me know if I can help you with any other requests.";
   private String prompt = "Please limit the responses to known facts. If you do not know about the context or the answer, simply say '" + ChatRequest.default
  public String getMessage() {
      return prompt + " " + message;
   }
  public void setMessage(String message) {
      this.message = prompt + " " + message;
   }
}
```

You have created a placeholder for the defaultMessage and used it inside the prompt. This variable will later be reused in the service to check the response from the LLM, and if it has this default message instead of calling extractor, you will send a default class instance with null values to keep it simple for now. Let's see this change in action.

File: ./src/main/java/com/example/travelx/ServiceWithPersistentMemory.java

```
package com.example.travelx;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import dev.langchain4j.model.openai.OpenAiChatModel;
import dev.langchain4j.service.AiServices;
@Configuration
public class ServiceWithPersistentMemory {
   private Assistant assistant;
   private OpenAiChatModel chatModel;
   @Bean
   public Assistant chatAssistant(Assistant assistant, OpenAiChatModel chatModel) {
      this.assistant = assistant;
      this.chatModel = chatModel;
      return this.assistant;
   }
   public Aircraft chat(String message) {
      String response = this.assistant.chat(message);
      Aircraft aircraft = new Aircraft();
      if (!response.equals(ChatRequest.defaultMessage)) {
         AircraftExtractor aircraftExtractor = AiServices.create(AircraftExtractor.class, chatModel);
         aircraft = aircraftExtractor.extractAircraftFrom(response);
      }
      return aircraft;
   }
   public Flight flightChat(String message) {
      String response = this.assistant.chat(message);
      Flight flight = new Flight();
      if (!response.equals(ChatRequest.defaultMessage)) {
         FlightExtractor flightExtractor = AiServices.create(FlightExtractor.class, chatModel);
         flight = flightExtractor.extractFlightFrom(response);
      }
      return flight;
   }
}
```

So now you have created a placeholder variable for the aircraft and flight to store the default instance object and check if the response from the LLM is equal to our previously created defaultMessage. If it is, you return it; otherwise, you extract the respective object entity from the response.

Let's reboot the application and test this.

```
curl -X POST http://localhost:8080/api/flight-chat -H "Content-Type: application/json" -d '{
```

```
 "message": "Tell me about flight details for AABB101."
}'
```

Output:

```
{"aircraft":null,"dateOfDeparture":null,"fromCity":null,"toCity":null,"departureAirport":null,"arrivalAirport":null,"flightNumber":null,"departureTime":null,"arriva
```

RAG Paradigm

Until this point, you have been using a simple query pattern, as depicted in Figure 7-2 where you are relying on the information of the LLM to answer the user's query.

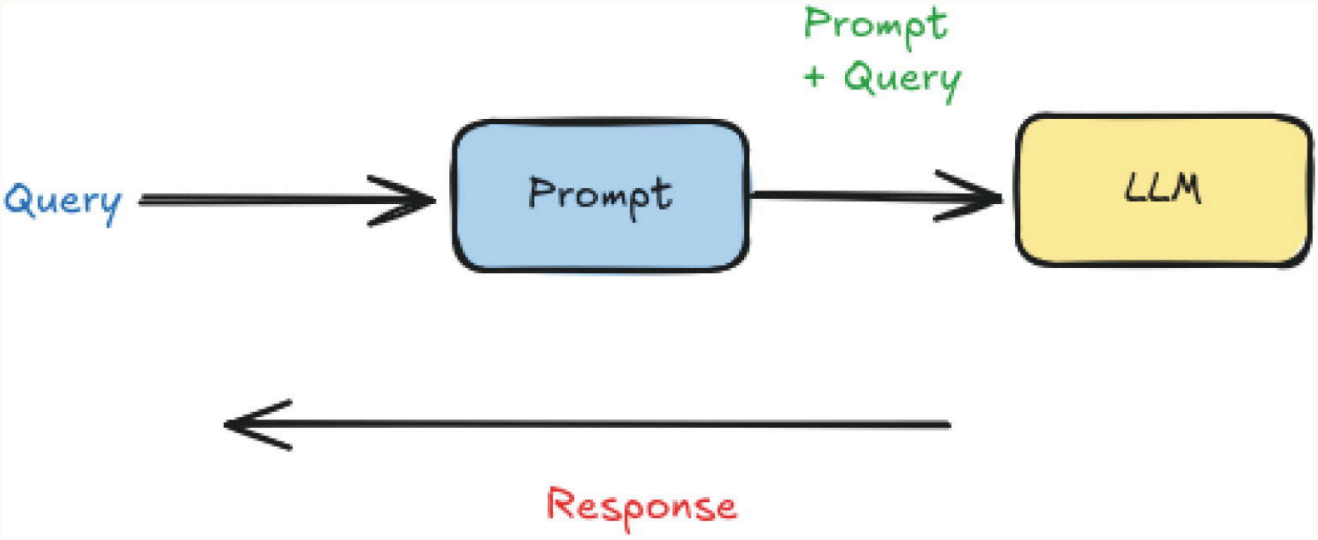


Figure 7-2 Simple query pattern

In addition, you have been able to suggest/advise the LLM to consider any rules, facts, or style in which you want it to reply in. Through prompt engineering, you can ask the LLM not to hallucinate through keeping its replies limited to its knowledge base. If you want the LLM to have more knowledge, you have to either rely on creating another foundation model or fine-tune an existing foundational model.

Training a foundational model means creating an LLM from scratch by feeding it large amounts of data. This takes a lot of time, resources in terms of effort, and GPUs costing a lot of money over several days. This method is sometimes out of reach for many organizations and startups that want to leverage LLM. There is another way to supplement the knowledge of existing foundational models, which is training them with additional data specific to your organization, which is effective in terms of cost and training resources and time. This requires some specialized knowledge in terms of building machine learning models using transformers and other architectures, which is out of scope of this book.

There is another cost-effective method in terms of achieving similar results without creating a foundational model from scratch or fine-tuning it. This methodology uses prompt engineering and the token size of the LLM into consideration to solve this problem statement. It's called the RAG paradigm.

RAG stands for retrieval-augmented generation. Let's try to understand it in simple incremental terms. Every LLM/SLM has a token space as the maximum number of tokens it can facilitate in its request-response cycle referred to as its *context length*. The concept of a token is a bit more complex than just considering it as a character, but for this discussion let's assume a token is a character.

Tokens are the fundamental units of text that LLMs use to understand and generate language. A token might be a single character, part of a word, a whole word, or even punctuation—depending on how the model's tokenizer breaks down the input. For example, the word "airplane" might be one token in some models but split into two or more tokens in others. LLMs do not process raw text directly; instead, they convert it into a sequence of tokens before performing any operation. Each model has a fixed *context length*, which is the maximum number of tokens it can handle in a single request, including both the input (prompt and context) and the output (response). If the total number of tokens exceeds this limit, part of the input or output may be truncated, leading to incomplete results. Efficient token management helps ensure that important context is retained and meaningful responses are generated. This is particularly important in tasks like RAG, where additional context is appended to prompts. Understanding how tokens work enables better prompt design and improves the reliability of responses from the LLM.

Let's take the example of the Deepseek-R1 model, which we used in previous chapters.

As shown in Figure 7-3, the context length for the R1 model is 128K tokens. This includes the request tokens plus the response tokens.

DeepSeek-R1 Models

Model	#Total Params	#Activated Params	Context Length	Download
DeepSeek-R1-Zero	671B	37B	128K	🤗 HuggingFace
DeepSeek-R1	671B	37B	128K	🤗 HuggingFace

Figure 7-3 Context length of DeepSeek R1 models(Reference: https://github.com/deepseek-ai/DeepSeek-R1)

The request token could be the sum of your customer query plus the conversation history memory tokens plus the prompt message you include to keep the context limited to facts, while the response token is the response from the LLM to answer the prompt request, as shown in Figure 7-4. This context length is specific to the LLM in use. Small foundational models have a smaller context length, while LLMs might have a bigger context length. Context length is helpful in two ways.

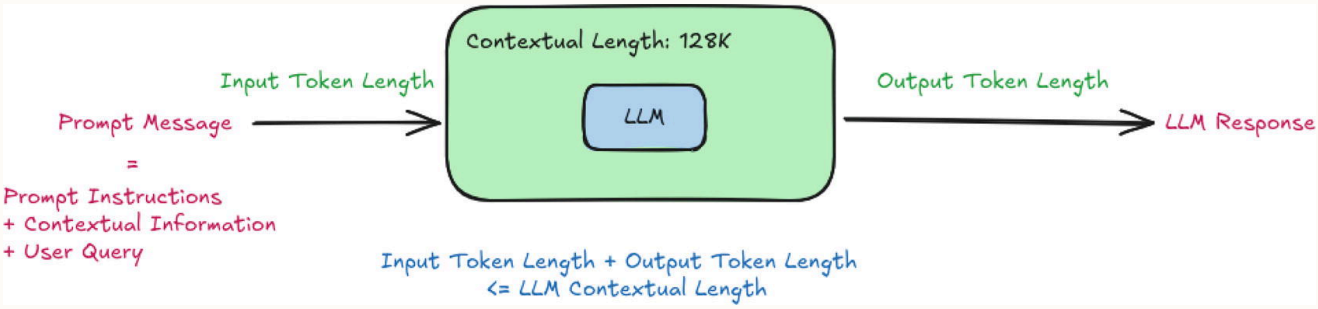


Figure 7-4 Contextual length and token summary for Deepseek-R1 context length

First, it is helpful when you are expecting a larger token response from the LLM and it can do so only if it has a larger contextual length by default and you do not take too much of this space through the use of the input tokens, which would reduce the output token length. Second, it is helpful when you are expecting to provide a huge context in the input token you provide it through prompt engineering, which could contain information related to your organization so that the LLM can use this context then to answer your query. Using more token space than the context length will truncate the extending tokens from the output response of the LLM. This will happen in two cases, all of which finally result in overflowing the contextual length.

The first case is when the input prompt message is greater than the contextual length. This results in two side effects. Since the input is greater in size, it is truncated in terms of how many tokens reach the LLM, and since the input is truncated, you are not sure if the LLM received the right question and which part of the prompt was truncated. It depends how you organized your prompt in terms of concatenating the instructions, context, and user query. And since there is no space left for any response in the context length, you will not receive any response in this case.

The second case is when the output that the LLM comes up with is larger than the remaining space of the Context Length - Prompt context length. So, the response might be 60K tokens, but the remaining contextual space might be 40K in which case you will receive only the 40K token length of the previously determined output from the LLM. This will give you an incomplete response.

So eventually token planning and management are crucial aspects of LLM engineering. Now that you understand the concept of contextual length, you can use this to understand how you can share contextual knowledge with the LLM. Let's say you have a document that has all of your organizational information that might be relevant to your customers and it results in a 50K token size. If the context length of the LLM facilitates the input prompt including this 50K token from the organizational contextual knowledge and the response you are expecting, you are good to provide the LLM with more information that it can use to answer the query of the users since it has this information to answer from in the prompt message. So with each prompt message to the LLM, you will always be sending the organization contextual information, as shown in Figure 7-5.

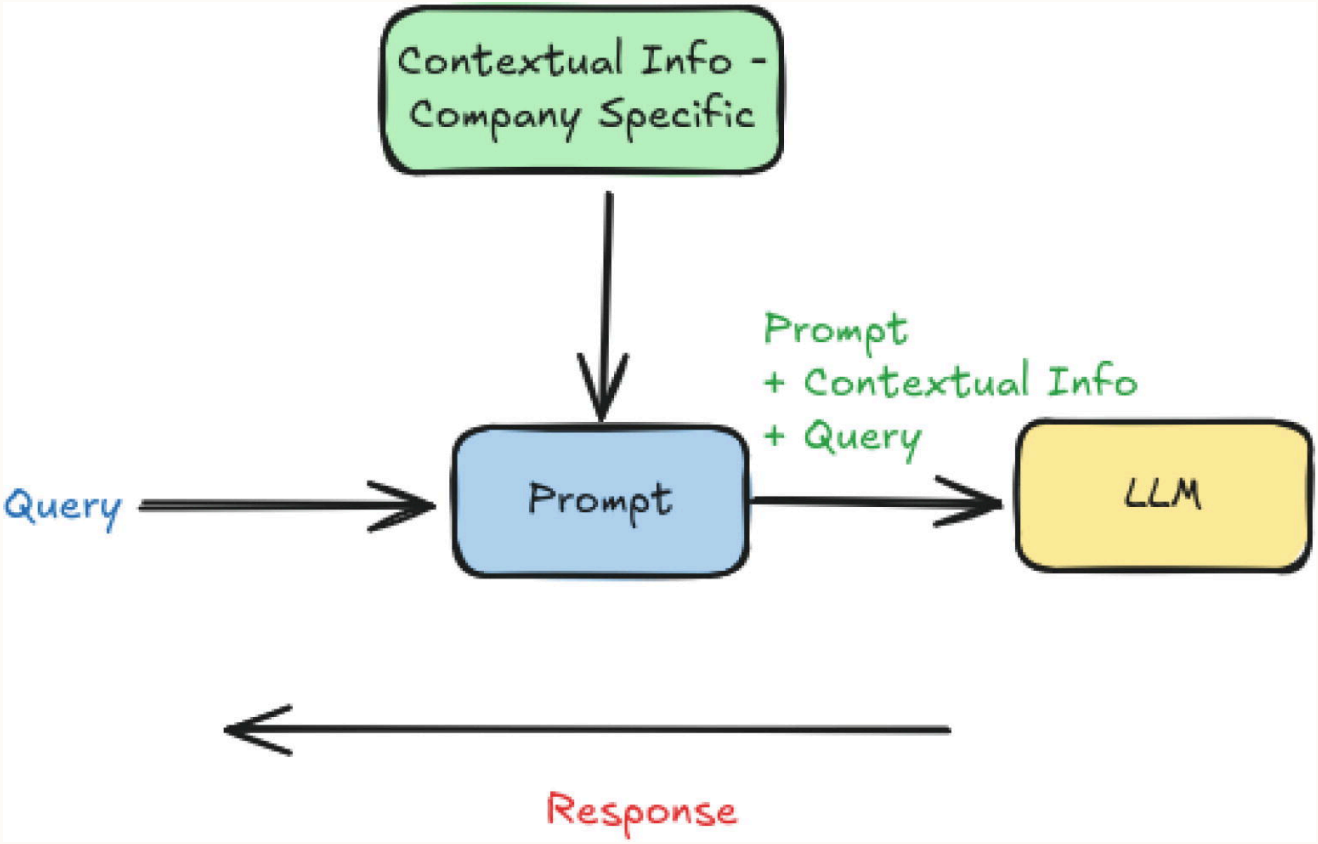


Figure 7-5 Contextual info in prompt

This may be fine as your organization is just starting but may not be feasible as you grow. For larger organizations, this data may be in several sources like SharePoint, PDFs, and databases and you cannot bring all of this information and always pass it to the prompt since it will in most cases be more than the context length. Also, not all of the information is helpful in answering the user's query. The whole organization might have information for different contexts like human resources, customers, and operations, while the user query might be relevant only to the operational aspect of the company. So, fetching all the info and feeding it to the LLM might be inefficient and will add to the input-output request-response time while you want to serve requests and solve their queries faster.

What if you could filter out the contextual information that is relevant only to your customer query before passing this information to the LLM? This not only results in less information but saves you contextual token length while being more specific in your response. So, you need a way to do a fast contextual search of your information/documents/resources and then use this information to answer the query of the customer in a natural language friendly manner using an LLM. This is what the RAG methodology does: retrieve (R) contextual information, (A) augment the knowledge of the foundational model, and (G) generate a natural language response to the query of the user using LLMs, as shown in Figure 7-6.

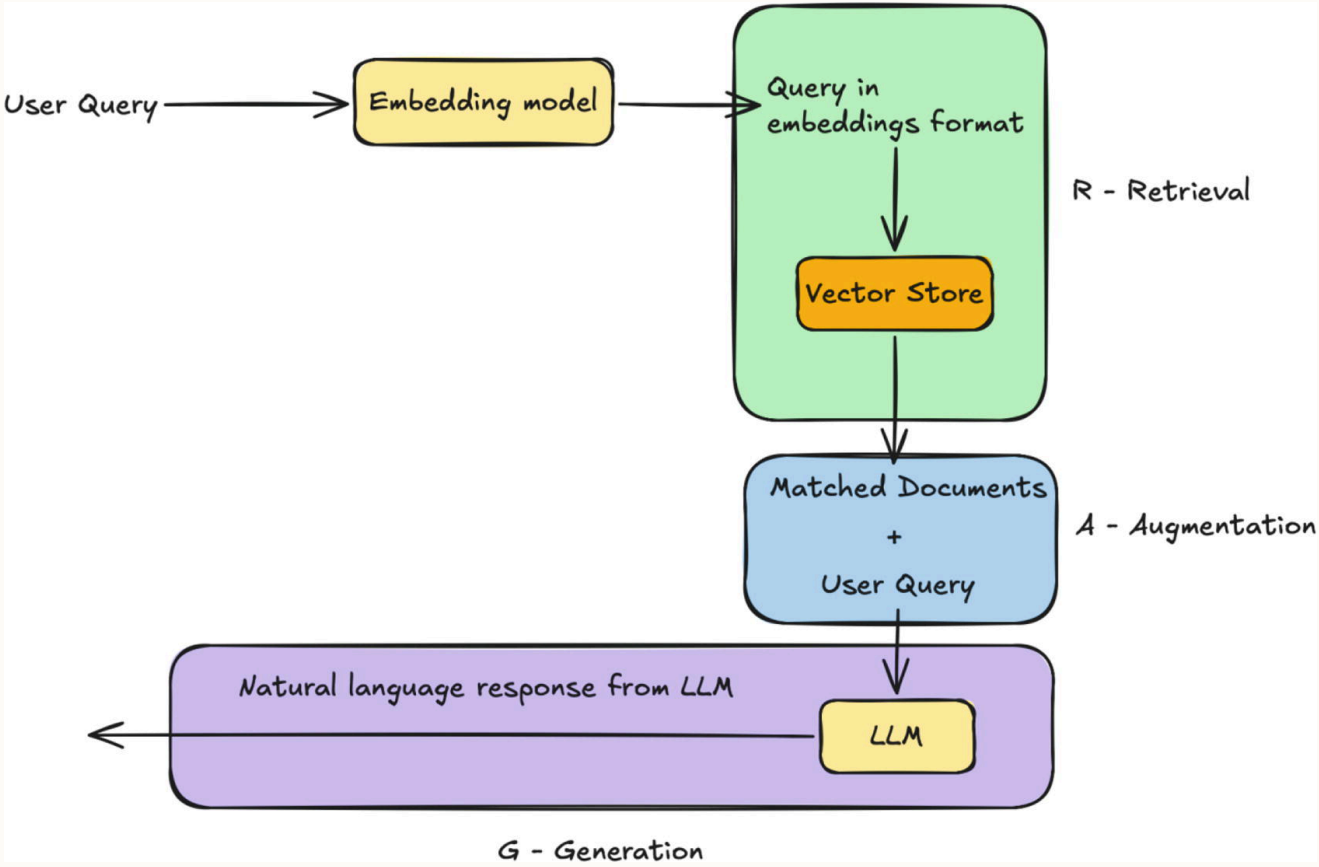


Figure 7-6 RAG paradigm

Embeddings

Embeddings are like magic stickers you put on words and pictures. Each sticker has a special number that tells you how similar things are. Imagine you have stickers for animals—dogs get stickers with numbers close to cats, because they're both pets and furry. But a fish would have a sticker with a number far away, because it's different. These magic stickers help computers quickly find things that go together, like matching questions with the best answers!

In the RAG paradigm, *embeddings* are numerical representations of data such as text, speech, images, or videos. Embeddings transform high-dimensional, heterogeneous real-world data into low-dimensional numerical vectors, capturing semantic meaning in a compact form. The geometric distance between these vectors reflects semantic similarity, allowing quick identification of contextually relevant information. For example, words or phrases with similar meanings have embeddings close to each other, enabling efficient retrieval of related documents or data chunks from a large dataset. By embedding both queries and data into the same vector space, RAG leverages vector databases to rapidly retrieve the most relevant contextual information, significantly reducing hallucinations and improving the accuracy of language model responses.

In RAG, embeddings are numerical representations of textual information used to efficiently retrieve relevant context for answering user queries. Embeddings convert words, sentences, or entire documents into vector representations within a high-dimensional space, such that texts with similar meanings or contexts placed closer together, while dissimilar texts are spaced farther apart. Within RAG, embeddings play a critical role by encoding organizational documents, knowledge bases, or databases into these numeric vectors. When a user submits a query, it is also converted into an embedding vector and compared against stored embeddings to swiftly retrieve the most relevant contextual information. This retrieved context is then appended to the query and provided to the language model, significantly reducing hallucinations and improving the accuracy and specificity of the generated response. For example, if a user asks about "flight AA101," their query embedding will closely match stored embeddings of actual flight details, allowing the model to provide precise, context-specific information. Overall, embeddings enable faster retrieval, minimize incorrect information, and enhance the scalability and reliability of answers provided by large language models within the RAG framework.

Vector Store

Imagine you have a special toy box that helps you find your toys really fast. Each toy has a magic sticker with numbers, and toys that are alike, like cars or dolls, have stickers with numbers close to each other. This special toy box is called a *vector store*, and it keeps all your toys organized by their magic stickers.

When you want a certain toy, you show the box another magic sticker that looks like the toy you want. The box quickly searches and finds the toys with stickers closest to the one you showed it—that's called *vector search*. So, instead of searching everywhere, the magic

box finds your favorite toys quickly because it knows exactly where similar toys are.

Traditional search methods, such as keyword search and full-text search, rely on matching exact words or phrases within documents. While useful, they fall short when the language used in a query doesn't exactly match the content in the database. For instance, searching for "morning coffee drink" might miss documents that mention only "cappuccino," even though they're semantically related. These traditional systems struggle with synonyms, misspellings, and varied phrasing.

To address this, we've moved toward vector search, which is based on semantic understanding rather than exact text matching. In vector search, both the data (documents, images, etc.) and the user's query are converted into embeddings—numerical vectors that capture the meaning of the content. Once both are in the same vector space, similarity between them can be calculated using mathematical metrics like cosine similarity, Euclidean distance, or dot product. This allows the system to return relevant results even if they use different words than the query.

A *vector store* (or *vector database*) is a specialized database designed to store and manage these embeddings efficiently. It enables high-speed, scalable similarity search across millions of data points. When a query is made, its embedding is compared to those in the vector store, and the closest (i.e., most semantically similar) vectors are retrieved—enabling much more relevant and intelligent results than keyword search.

Some popular open-source vector stores include:

- FAISS (Facebook AI Similarity Search)
- Weaviate
- ChromaDB
- Annoy
- Milvus

There are also managed and enterprise-ready vector search solutions like:

- Google Vertex AI Vector Search
- Pinecone
- ElasticSearch with vector search plugins
- AlloyDB (with pgvector and ScaNN)

These vector stores often support hybrid search, combining both vector (semantic) and keyword (literal) search capabilities for best-of-both-worlds retrieval.

The evolution from keyword to vector search reflects a broader shift toward meaning-based search, where understanding the user's intent and context is more important than exact word matches. This shift is especially critical in AI-driven applications like recommendation systems, fraud detection, question answering, and RAG.

RAG in Action

You will start by using a feature of LangChain4j called easy RAG to get started with the RAG pattern without using any subcomponents involved.

Let's update a dependency in our build.gradle file.

File: ./build.gradle

```
dependencies {
    ...
    implementation 'dev.langchain4j:langchain4j-easy-rag:1.0.0-alpha1'
    ...
}
```

Being an airline company, the example company has proprietary flight data. This could be present in any data store and format like CSV, JSON, or relational databases. To keep it simple, you will store all the flight data in a text file in CSV format. The first row will have field names like Aircraft, Date of Departure, From/To, etc., and the subsequent rows will have different flight details. This file can be stored in the resources folder.

File: ./src/main/resources/static/flight-details.txt

```
Aircraft,Date of Departure,From (US City),To (US City),Departure Airport,Arrival Airport,Flight Number,Departure Time,Arrival Time,Status
Boeing 737,2023-11-01,New York,Los Angeles,JFK,LAX,AA101,08:00,11:00,On Time
Airbus A320,2023-11-01,Chicago,Miami,ORD,MIA,DL202,09:30,13:00,On Time
Boeing 787,2023-11-01,San Francisco,Seattle,SFO,SEA,UA303,10:15,12:00,Delayed
Airbus A321,2023-11-01,Houston,Dallas,IAH,DFW,SW404,11:45,13:15,On Time
Boeing 737,2023-11-01,Atlanta,Denver,ATL,DEN,AA505,12:30,15:00,On Time
Airbus A320,2023-11-01,Boston,Las Vegas,BOS,LAS,DL606,14:00,17:30,Cancelled
Boeing 787,2023-11-01,Philadelphia,Phoenix,PHL,PHX,UA707,15:45,18:30,On Time
Airbus A321,2023-11-01,Orlando,San Diego,MCO,SAN,SW808,16:30,19:00,On Time
Boeing 737,2023-11-02,New York,Chicago,JFK,ORD,AA909,07:00,09:00,On Time
Airbus A320,2023-11-02,Los Angeles,Houston,LAX,IAH,DL101,08:30,12:00,On Time
Boeing 787,2023-11-02,Miami,Atlanta,MIA,ATL,UA202,10:00,12:00,Delayed
Airbus A321,2023-11-02,Seattle,Dallas,SEA,DFW,SW303,11:15,14:30,On Time
Boeing 737,2023-11-02,Denver,Boston,DEN,BOS,AA4041,13:00,17:00,On Time
Airbus A320,2023-11-02,Las Vegas,Philadelphia,LAS,PHL,DL505,14:30,18:00,Cancelled Boeing 787,2023-11-02,Phoenix,Orlando,PHX,MCO,UA606,16:00,20:00
Airbus A321,2023-11-02,San Diego,New York,SAN,JFK,SW707,17:45,21:30,On Time
Boeing 737,2023-11-03,Chicago,Los Angeles,ORD,LAX,AA808,08:00,10:30,On Time
Airbus A320,2023-11-03,Houston,San Francisco,IAH,SFO,DL909,09:30,12:00,On Time
Boeing 787,2023-11-03,Atlanta,Seattle,ATL,SEA,UA101,11:00,14:00,Delayed
Airbus A321,2023-11-03,Dallas,Miami,DFW,MIA,SW202,12:15,15:30,On Time
Boeing 737,2023-11-03,Boston,Denver,BOS,DEN,AA303,13:30,16:30,On Time
Airbus A320,2023-11-03,Philadelphia,Las Vegas,PHL,LAS,DL404,15:00,18:30,Cancelled Boeing 787,2023-11-03,Orlando,Phoenix,MCO,PHX,UA505,16:45,20:00,
Airbus A321,2023-11-03,New York,San Diego,JFK,SAN,SW606,18:00,21:30,On Time
Boeing 737,2023-11-04,Los Angeles,Chicago,LAX,ORD,AA707,07:30,10:00,On Time
Airbus A320,2023-11-04,San Francisco,Houston,SFO,IAH,DL808,09:00,12:30,On Time
Boeing 787,2023-11-04,Seattle,Atlanta,SEA,ATL,UA909,10:30,13:30,Delayed
Airbus A321,2023-11-04,Miami,Dallas,MIA,DFW,SW101,11:45,14:45,On Time
Boeing 737,2023-11-04,Denver,Boston,DEN,BOS,AA202,13:00,17:00,On Time
Airbus A320,2023-11-04,Las Vegas,Philadelphia,LAS,PHL,DL303,14:30,18:00,Cancelled Boeing 787,2023-11-04,Phoenix,Orlando,PHX,MCO,UA404,16:00,20:00
Airbus A321,2023-11-04,San Diego,New York,SAN,JFK,SW505,17:45,21:30,On Time
Boeing 737,2023-11-05,Chicago,Los Angeles,ORD,LAX,AA606,08:00,10:30,On Time
Airbus A320,2023-11-05,Houston,San Francisco,IAH,SFO,DL707,09:30,12:00,On Time
Boeing 787,2023-11-05,Atlanta,Seattle,ATL,SEA,UA808,11:00,14:00,Delayed
Airbus A321,2023-11-05,Dallas,Miami,DFW,MIA,SW909,12:15,15:30,On Time
Boeing 737,2023-11-07,Chicago,Los Angeles,ORD,LAX,AA404,08:00,10:30,On Time
Airbus A320,2023-11-07,Houston,San Francisco,IAH,SFO,DL505,09:30,12:00,On Time
Boeing 787,2023-11-07,Atlanta,Seattle,ATL,SEA,UA606,11:00,14:00,Delayed
Airbus A321,2023-11-07,Dallas,Miami,DFW,MIA,SW707,12:15,15:30,On Time
Boeing 737,2023-11-08,Los Angeles,Chicago,LAX,ORD,AA303,07:30,10:00,On Time
Airbus A320,2023-11-08,San Francisco,Houston,SFO,IAH,DL404,09:00,12:30,On Time
Boeing 787,2023-11-08,Seattle,Atlanta,SEA,ATL,UA505,10:30,13:30,Delayed
Airbus A321,2023-11-08,Miami,Dallas,MIA,DFW,SW606,11:45,14:45,On Time
```

You will then extend the LangchainConfig.java file from earlier to create an in-memory embedding store that will store the document content for later use. Let's define a private method that parses the text content from the resources folder and then loads the documents into the document store.

```
private InMemoryEmbeddingStore<TextSegment> embeddingStore() {
    List<Document> documents = FileSystemDocumentLoader.loadDocuments("/path/to/your/travelx/src/main/resources/static", new TextDocumentParser());
    InMemoryEmbeddingStore<TextSegment> embeddingStore = new InMemoryEmbeddingStore<>();
    EmbeddingStoreIngestor.ingest(documents, embeddingStore);
    return embeddingStore;
}
```

You will now use the embeddingStore in the search logic by using the contentRetriever method, which references the embedding store.

```
@Bean
public Assistant openAiAssistant() {
    this.chatMemory = this.messageWindowChatMemory();
    this.model = this.openAiChatModel();
    this.assistant = AiServices.builder(Assistant.class)
            .chatLanguageModel(this.model)
            .chatMemory(this.chatMemory)
            .contentRetriever(EmbeddingStoreContentRetriever.from(this.embeddingStore()))
            .build();
    return this.assistant;
```

```
}
```

The complete code changes necessary have been shared below.

File: ./src/main/java/com/example/travelx/LangchainConfig.java

```
import dev.langchain4j.data.document.Document;
import dev.langchain4j.data.document.loader.FileSystemDocumentLoader;
import dev.langchain4j.data.document.parser.TextDocumentParser;
import dev.langchain4j.data.segment.TextSegment;
import dev.langchain4j.rag.content.retriever.EmbeddingStoreContentRetriever;
import dev.langchain4j.store.embedding.EmbeddingStoreIngestor;
import dev.langchain4j.store.embedding.inmemory.InMemoryEmbeddingStore;
private InMemoryEmbeddingStore<TextSegment> embeddingStore() {
   List<Document> documents = FileSystemDocumentLoader.loadDocuments("/path/to/your/travelx/src/main/resources/static", new TextDocumentParser()
   InMemoryEmbeddingStore<TextSegment> embeddingStore = new InMemoryEmbeddingStore<>();
   EmbeddingStoreIngestor.ingest(documents, embeddingStore);
   return embeddingStore;
}
@Bean
public Assistant openAiAssistant() {
   this.chatMemory = this.messageWindowChatMemory();
   this.model = this.openAiChatModel();
   this.assistant = AiServices.builder(Assistant.class)
         .chatLanguageModel(this.model)
         .chatMemory(this.chatMemory)
         .contentRetriever(EmbeddingStoreContentRetriever.from(this.embeddingStore()))
         .build();
   return this.assistant;
}
```

With this simple change, you are able to set up a contextual RAG pipeline. Let's start the application and test this.

```
curl -X POST http://localhost:8080/api/flight-chat -H "Content-Type: application/json" -d '{
 "message": "Tell me about flight details for AA4041."
 Boeing 737,2023-11-02,Denver,Boston,DEN,BOS,AA4041,13:00,17:00,On Time
}'
```

Output:

```
{"aircraft":"Boeing 737","dateOfDeparture":"2023-11-02","fromCity":"Denver","toCity":"Boston","departureAirport":"DEN","arrivalAirport":"BOS","flightNumber
```

Now, you are able to see your flight details being fetched from your internal data source.

Conclusion

In this chapter, you explored how to integrate contextual information into your LLM ecosystem using the LangChain4j framework and the RAG paradigm. Initially, the application suffered from hallucinations due to the limitations of generalized LLM responses. To address this, you implemented prompt engineering and context-awareness checks within the Spring Boot application, significantly reducing the incorrect outputs. You then used the RAG approach to leverage embeddings and vector stores to retrieve specific organizational data and augment the LLM's knowledge base. By integrating the easy RAG module of LangChain4j, our application could efficiently provide accurate and context-specific answers based on internal documents and proprietary data sources. This enhancement not only improved response accuracy but also enabled the provision of more reliable and organization-specific information to our customers.

For readers interested in exploring these concepts further, consider referring to the official documentation for LangChainjJ, foundational materials on RAG, introductory guides on vector databases like FAISS or Weaviate, and technical overviews of embeddings and tokenization strategies provided by major LLM platforms such as OpenAI or Hugging Face. These resources offer valuable insights into building scalable and context-aware AI applications using modern LLM tooling.

And now, you will move on to integrating the Spring AI library.

Previous chapter
< 6. Integrating LLMs with Java Applications

Next chapter
8. Spring AI Ninja Moves >